PNAS Nexus, 2024, 3, 1–10
https://doi.org/10.1093/pnasnexus/pgae063
Advance access publication 9 February 2024
Research Report

Inferring networks from time series: A neural approach

Thomas Gaskin [a,b,*], Grigorios A. Pavliotis [b] and Mark Girolami [c,d]

[a]Department of Applied Mathematics and Theoretical Physics, University of Cambridge, Cambridge CB3 0WA, UK
[b]Department of Mathematics, Imperial College London, London SW7 2AZ, UK
[c]Department of Engineering, University of Cambridge, Cambridge CB2 1PZ, UK
[d]The Alan Turing Institute, London NW1 2DB, UK
*To whom correspondence should be addressed: Email: trg34@cam.ac.uk
Edited By: Derek Abbott

Abstract

Network structures underlie the dynamics of many complex phenomena, from gene regulation and foodwebs to power grids and social media. Yet, as they often cannot be observed directly, their connectivities must be inferred from observations of the dynamics to which they give rise. In this work, we present a powerful computational method to infer large network adjacency matrices from time series data using a neural network, in order to provide uncertainty quantification on the prediction in a manner that reflects both the degree to which the inference problem is underdetermined as well as the noise on the data. This is a feature that other approaches have hitherto been lacking. We demonstrate our method's capabilities by inferring line failure locations in the British power grid from its response to a power cut, providing probability densities on each edge and allowing the use of hypothesis testing to make meaningful probabilistic statements about the location of the cut. Our method is significantly more accurate than both Markov-chain Monte Carlo sampling and least squares regression on noisy data and when the problem is underdetermined, while naturally extending to the case of nonlinear dynamics, which we demonstrate by learning an entire cost matrix for a nonlinear model of economic activity in Greater London. Not having been specifically engineered for network inference, this method in fact represents a general parameter estimation scheme that is applicable to any high-dimensional parameter space.

Keywords: network inference, neural differential equations, model calibration, power grids

Significance Statement

In this work, we learn static network structures from time series, an important problem across the quantitative sciences, where networks can appear either as physical links or abstract connections, but in many cases will only be indirectly observable through their dynamics. We show our method to be more accurate, computationally efficient, and versatile both than Markov-chain Monte Carlo sampling and regression. Additionally, it provides meaningful uncertainty quantification on the network prediction, which is both novel and key, as it allows estimating the range of networks compatible with the observation data, which may not fully determine the inference problem. Our method thus lets researchers make probabilistic statements about the connectivity matrices underlying the dynamics they observe.

Introduction

Networks are important objects of study across the scientific disciplines. They materialize as physical connections in the natural world, for instance as the mycorrhizal connections between fungi and root networks that transport nutrients and warning signals between plants (1, 2), human traffic networks (3, 4), or electricity grids (5, 6). However, they also appear as abstract, nonphysical entities, such as when describing biological interaction networks and food webs (7–9), gene or protein networks (10–13), economic cost relations (14, 15), or social links between people along which information (and misinformation) can flow (16–18). In all examples, though the links constituting the network may not be tangible, the mathematical description is the same. In this work, we are concerned with inferring the structure of a static network from observations of dynamics on it. The problem is of great scientific bearing: for instance, one may wish to understand the topology of an online social network from observing how information is passed through it, and some work has been done on this question (19–21). Another important application is inferring the connectivity of neurons in the brain by observing their responses to external stimuli (22, 23). In an entirely different setting, networks crop up in statistics in the form of conditional independence graphs, describing dependencies between different variables, which again are to be inferred from data (24, 25).

Our approach allows inferring network connectivities from time series data with uncertainty quantification. Uncertainty quantification for network inference is important for two reasons: first, the observations will often be noisy, and one would like the uncertainty on the data to translate to the uncertainty on the predicted

Competing Interest: The authors declare no competing interest.
Received: May 18, 2023. **Accepted:** January 22, 2024
© The Author(s) 2024. Published by Oxford University Press on behalf of National Academy of Sciences. This is an Open Access article distributed under the terms of the Creative Commons Attribution License (https://creativecommons.org/licenses/by/4.0/), which permits unrestricted reuse, distribution, and reproduction in any medium, provided the original work is properly cited.

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

network. Second, however, completely inferring large networks requires equally large amounts of data—typically at least $N-1$ equations per node, N being the number of nodes—and these observations must furthermore be linearly independent. Data of such quality and quantity will often not be available, leading to an underdetermined inference problem. The uncertainty on the predicted network should thus also reflect (at least to a certain degree) the "nonconvexity" of the loss function, i.e. how many networks are compatible with the observed data. To the best of our knowledge, no current network inference method is able to provide this information.

Network inference can be performed using ordinary least squares (OLS) regression (6, 26), but this is confined to the case where the dynamics are linear in the adjacency matrix. An alternative are sampling-based methods that generalize to the nonlinear case (27–29), but these tend to struggle in very high-dimensional settings and can be computationally expensive. Efficient inference methods for large networks have been developed for cascading dynamics (19–21), but these are highly specialized to a particular type of observation data and give no uncertainty quantification on the network prediction. Our method avoids these limitations. Its use of neural networks is motivated by their recent and successful application to low-dimensional parameter calibration problems (30, 31), both on synthetic and real data, as well as by their conceptual proximity to Bayesian inference, e.g. through neural network Gaussian processes or Bayesian neural networks (32–37). Our method's underlying approach ties into this connection, and in fact, since it has not been specifically engineered to fit the network case, constitutes a general and versatile parameter estimation method.

Method description

We apply the method proposed in Ref. (31) to the network case. The approach consists of training a neural network to find a graph adjacency matrix $\hat{\mathbf{A}} \in \mathbb{R}^{N\times N}$ that, when inserted into the model equations, reproduces the observed time series $\mathbf{T} = (\mathbf{x}_1, \ldots, \mathbf{x}_L)$. A neural network is a function $u_\theta : \mathbb{R}^{N\times q} \to \mathbb{R}^p$, where $q \geq 1$ represents the number of time series steps that are passed as input. Its output is the (vectorized) estimated adjacency matrix $\hat{\mathbf{A}}$, which is used to run a numerical solver for B iterations (B is the batch size) to produce an estimated time series $\hat{\mathbf{T}}(\hat{\mathbf{A}}) = (\hat{\mathbf{x}}_i, \ldots, \hat{\mathbf{x}}_{i+B})$. This in turn is used to train the internal parameters $\boldsymbol{\theta}$ of the neural net (the weights and biases) via a loss function $J(\hat{\mathbf{A}} \mid \mathbf{T})$. The likelihood of any sampled estimate is simply proportional to

$$p(\hat{\mathbf{A}} \mid \mathbf{T}) \propto e^{-J} \tag{1}$$

and by Bayes' rule, the posterior density is then

$$\pi(\hat{\mathbf{A}} \mid \mathbf{T}) = p(\hat{\mathbf{A}} \mid \mathbf{T}) \times \pi^0(\hat{\mathbf{A}}), \tag{2}$$

with π^0 the prior density (38). As $\hat{\mathbf{A}} = \hat{\mathbf{A}}(\boldsymbol{\theta})$, we may calculate the gradient $\nabla_{\boldsymbol{\theta}} J$ and use it to optimize the internal parameters of the neural net using a backpropagation method of choice; popular choices include stochastic gradient descent, Nesterov schemes, or the Adam optimizer (39). Calculating $\nabla_{\boldsymbol{\theta}} J$ thus requires differentiating the predicted time series $\hat{\mathbf{T}}$, and thereby the system equations, with respect to $\hat{\mathbf{A}}$. In other words, the loss function contains knowledge of the dynamics of the model. Finally, the true data are once again input to the neural net to produce a new parameter estimate $\hat{\mathbf{A}}$, and the cycle starts afresh. A single pass over the entire dataset is called an epoch.

Using a neural net allows us to exploit the fact that, as the net trains, it traverses the parameter space, calculating a loss at each point. Unlike Monte Carlo sampling, the posterior density is not constructed from the frequency with which each point is sampled, but rather calculated directly from the loss value at each sample point. This entirely eliminates the need for rejection sampling or a burn-in: at each point, the true value of the likelihood is obtained, and sampling a single point multiple times provides no additional information, leading to a significant improvement in computational speed. Since the stochastic sampling process is entirely gradient-driven, the regions of high probability are typically found much more rapidly than with a random sampler, leading to a high sample density around the modes of the target distribution. We thus track the neural network's path through the parameter space and gather the loss values along the way. Multiple training runs can be performed in parallel, and each chain terminated once it reaches a stable minimum, increasing the sampling density on the domain, and ensuring convergence to the posterior distribution in the limit of infinitely many chains.

We begin this article with two application studies: first, we infer locations of a line failure in the British power grid from observations of the network response to the cut; and second, we infer economic cost relations between retail centers in Greater London. Thereafter, we conduct a comparative analysis of our method's performance, before finally demonstrating the connection between the uncertainty on the neural net prediction and the uncertainty of the inference problem.

Inferring line failures in the British power grid

Power grids can be modeled as networks of coupled oscillators using the Kuramoto model of synchronized oscillation (40–44). Each node i in the network either produces or consumes electrical power P_i while oscillating at the grid reference frequency Ω. The nodes are connected through a weighted undirected network $\mathbf{A} = (a_{ij})$, where the link weights $a_{ij} \sim Y_{ij}U_{ij}^2$ are obtained from the electrical admittances Y_{ij} and the voltages U_{ij} of the lines. The network coupling allows the phases $\varphi_i(t)$ of the nodes to synchronize according to the differential equation (43)

$$\alpha \frac{d^2\varphi_i}{dt^2} + \beta \frac{d\varphi_i}{dt} = P_i + \kappa \sum_j a_{ij} \sin(\varphi_j - \varphi_i), \tag{3}$$

where α, β, and κ are the inertia, friction, and coupling coefficients, respectively. A requirement for dynamical stability of the grid is that $\sum_i P_i = 0$, i.e. that as much power is put into the grid as is taken out through consumption and energy dissipation (42).

A power line failure causes the network to redistribute the power loads, causing an adjustment cascade to ripple through the network until equilibrium is restored (5). In this work, we recover the location of a line failure in the British power grid from observing these response dynamics. Figure 1a shows the high-voltage transmission grid of Great Britain as of January 2023, totalling 630 nodes (representing power stations, substations, and transformers) and 763 edges with their operating voltages. Of the roughly 1,300 power stations dotted around the island, we include those 38 with installed capacities of at least 400 MW that are directly connected to the national grid (45); following Refs. (5, 42), we give all other nodes a random value $P_i \sim \mathcal{U}[-200, +200]$ such that $\sum_i P_i = 0$.

We simulate a power cut in the northeast of England by iterating the Kuramoto dynamics until the system reaches a steady state of equilibrium (defined as $|\dot{\varphi}_i|/\varphi_i \leq 0.01\ \forall i$) and then

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

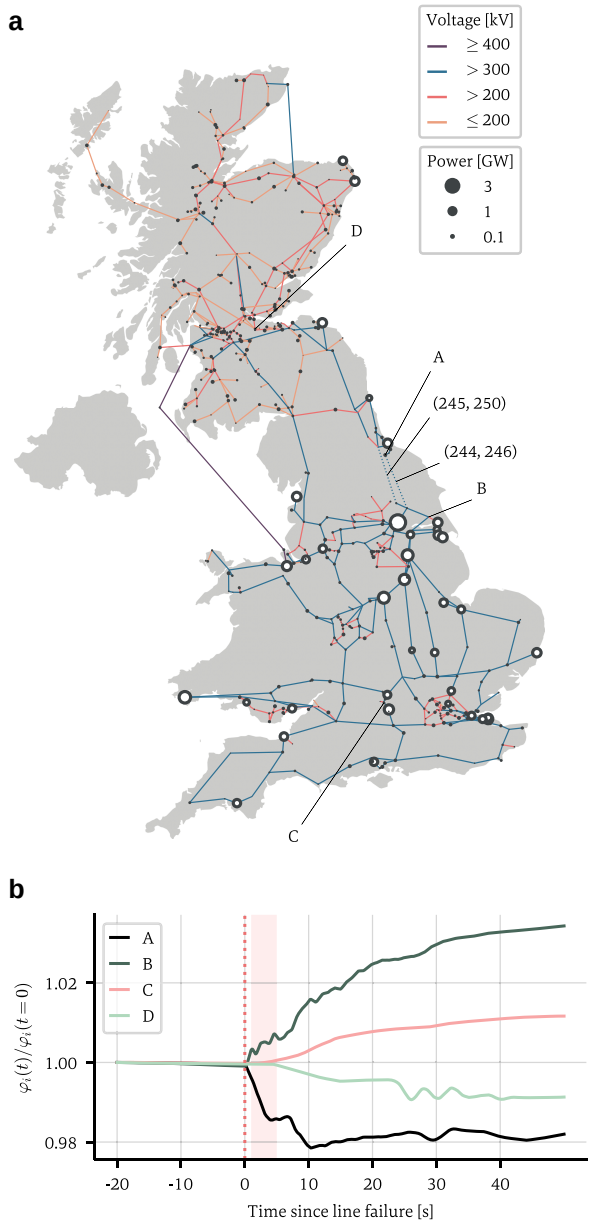

Fig. 1. a) Approximate high-voltage electricity transmission grid of Great Britain. Six hundred and thirty accurately placed nodes, representing power stations, substations, and transmission line intersections, and their connectivity as of January 2023 are shown (46–48). Colors indicate the operating voltage of the lines. The size of the nodes indicate their power generation or consumption capacity (absolute values shown). White ringed nodes indicate the 38 nodes that are real power stations with capacities over 400 MW (45), with all other nodes assigned a random capacity in $[-200, +200]$. The two dotted edges in the northeast of England are the edges affected by a simulated power cut, labeled by the indices of their start and end vertices. b) The network response to the simulated power line failure, measured at four different nodes in the network (marked A–D). The equation parameters were tuned to ensure phase-locking of the oscillators ($\alpha = 1, \beta = 0.2, \kappa = 60$). Nodes closer to the location of the line cut (A and B) show a stronger and more immediate response than nodes further away (C and D). The shaded area indicates the 4-second window we use to infer the line location. Background image: (49).

removing two links and recording the network response (Fig. 1b). From the response, we can infer the adjacency matrix of the perturbed network $\tilde{\mathbf{A}}$ (with missing links) and, by comparing with the unperturbed network $\mathbf{A}^0$ (without missing links), the line failure locations.

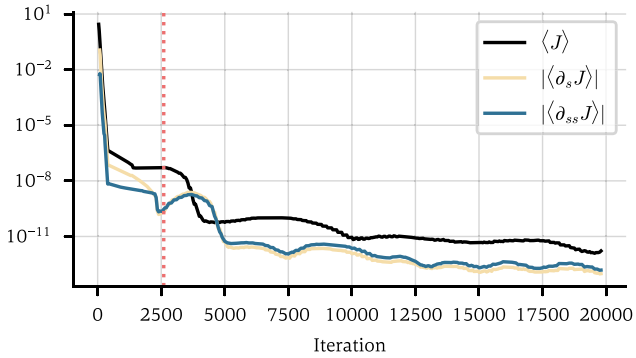

Fig. 2. The total loss J and its derivatives with respect to the iteration count $\partial_s J$ and $\partial_{ss} J$, averaged over a window of 20 iterations (absolute values shown). The dotted line indicates the value at which ν is set to 0.

We let a neural network output a (vectorized) adjacency matrix $\hat{\mathbf{A}}$ and use this estimated adjacency matrix to run the differential equation [3], which will produce an estimate $\hat{\mathbf{T}}$ of the observed time series of phases $\mathbf{T}$. A hyperparameter sweep on synthetic data showed that using a deep neural network with 5 layers, 20 nodes per layer, and no bias yields optimal results (see Figs. S2–S4). We use the hyperbolic tangent as an activation function on each layer except the last, where we use the "hard sigmoid" (50, 51)

$$\sigma(x) = \begin{cases} 0, & x \leq -3, \\ 1, & x \geq +3, \\ x/6 + 1/2, & \text{else}, \end{cases}$$

which allows neural net output components to actually become zero, and not just asymptotically close, thereby ensuring sparsity of the adjacency matrix—a reasonable assumption given that the power grid is far from fully connected. We use the Adam optimizer (39) with a learning rate of 0.002 for the gradient descent step. Since the neural network outputs are in [0, 1], we scale the network weights $a_{ij} \to \lambda a_{ij}$ such that $a_{ij} \in [0, 1]$, and absorb the scaling constant λ into the coupling constant κ; see supplementary material for details on the calculations.

We use the following loss function to train the internal weights $\boldsymbol{\theta}$ of the neural network such that it will output an adjacency matrix that reproduces the observed data:

$$J(\hat{\mathbf{A}} \mid \mathbf{T}) = \|\hat{\mathbf{T}}(\hat{\mathbf{A}}) - \mathbf{T}\|_2^2 + \|\hat{\mathbf{A}} - \hat{\mathbf{A}}^{\mathsf{T}}\|_2^2 + \mathrm{tr}(\hat{\mathbf{A}}) + \nu\|\hat{\mathbf{A}} - \mathbf{A}^0\|_2^2.$$

The first summand is the data-model mismatch, the second penalizes asymmetry to enforce undirectedness of the network, and the third sets the diagonal to zero (which cannot be inferred from the data, since all terms $\sin(\theta_j - \theta_i) = 0$ for $i = j$). $\nu = \nu(s)$ is a function of the iteration count s designed to let the neural network search for $\tilde{\mathbf{A}}$ in the vicinity of $\mathbf{A}^0$, since we can assume a priori that the two will be similar in most entries. To this end, we set $\nu = 10$ while the loss function has not yet reached a stable minimum, quantified by $|\langle \partial_s J \rangle| > 10^{-10}$ and $|\langle \partial_{ss} J \rangle| > 10^{-10}$, and $\nu = 0$ thereafter. Here, $\langle \cdot \rangle$ is a rolling average over a window of 20 iterations, see Fig. 2. In other words, we push the neural network toward a stable minimum in the neighborhood of $\mathbf{A}^0$ and, once the loss stabilizes, permanently set $\nu = 0$.

In theory $L = N - 1$, observations are needed to completely infer the network, though symmetries in the data usually mean $L > N$ is required in practice (52). In this experiment, we purposefully underdetermine the problem by only using $L < N - 1$ steps; additionally, we train the network on data recorded 1 simulated

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

second after the power cut, where many nodes will still be close to equilibrium. Although the neural network may be unable to completely infer the network, it can nevertheless produce a joint distribution on the network edge weights $p(\hat{\mathbf{A}} \mid \mathbf{T})$, recorded during the training, that allows us to perform hypothesis testing on the line failure location. The marginal likelihood on each network edge $\hat{a}_{ij}$ is given by

$$\rho(\hat{a}_{ij} \mid \mathbf{T}) = \int p(\hat{\mathbf{A}} \mid \mathbf{T}) \mathrm{d}\hat{\mathbf{A}}_{-ij} \times \pi^0(\hat{a}_{ij}), \tag{4}$$

where the $-ij$ subscript indicates we are omitting the ijth component of $\hat{\mathbf{A}}$ in the integration. We assume uniform priors π^0 on each edge. In high dimensions, calculating the joint of all network edge weights can become computationally infeasible, but we can circumvent this by instead considering the two-dimensional joint density of the edge weight under consideration and the likelihood, $p(\hat{a}_{ij}, e^{-J})$, and then integrating over the likelihood,

$$\rho(\hat{a}_{ij} \mid \mathbf{T}) = \int p\left(\hat{a}_{ij}, e^{-J}\right) \mathrm{d}(e^{-J}). \tag{5}$$

We show the results in Fig. 3. Given the marginal distributions $\rho(\hat{a}_{ij} \mid \mathbf{T})$ with modes $\tilde{a}_{ij}$, we plot the densities on the four network edges with the highest relative prediction error $|\tilde{a}_{ij} - a^0_{ij}|/a^0_{ij}$. The advantage of obtaining uncertainty quantification on the network is now immediately clear: even in the underdetermined case, we are able to make meaningful statistical statements about the line failure location. We see that the missing edges consistently have the highest relative prediction errors, and that the p-values for measuring the unperturbed value a^0_{ij} under the null $\hat{a}_{ij}$ are 0.2% and 0.04%, respectively, while being statistically insignificant for all other edges. It is interesting to note that the other candidate locations are also within the vicinity of the line failure, though their predicted values are much closer to the unperturbed value. In Fig. 3b, we see that the predicted network reproduces the response dynamics for the range covered by the training data when inserted into Eq. 3, but, since the problem was purposefully underdetermined, the errors in the prediction $\hat{\mathbf{A}}$ cause the predicted and true time series to diverge for larger t. Densities on all 200,000 potential edges were obtained in about 20 min on a regular laptop CPU.

Inferring economic cost networks from noisy data

In the previous example, the underlying network was a physical entity, but in many cases networks model abstract connections. We therefore now consider a commonly used economic model of the coupling of supply and demand (14, 15, 56) and a dataset of economic activity across Greater London. The goal is to learn the entire coupling network, not just to infer the (non)existence of individual edges. In the model, N origin zones of sizes O_i, representing economic demand, are coupled to M destination zones of sizes W_j, modeling the supply side, through a network whose weights quantify the convenience with which demand from zone i can be supplied from zone j: the higher the weight, the more demand flows through that edge (see Fig. 4a). Such a model is applicable e.g. to an urban setting (14), the origin zones representing residential areas, the destination zones e.g. commercial centers, and the weights quantifying the connectivity between the two (transport times, distances, etc.). The resulting cumulative demand at destination zone j depends both on the current size $W_j(t)$ of the destination zone and the network weights c_{ij}:

$$D_j = \sum_{i=1}^{N} \frac{W_j(t)^{\alpha} c_{ij}^{\beta}}{\sum_{k=1}^{M} W_k(t)^{\alpha} c_{ik}^{\beta}} O_i(t). \tag{6}$$

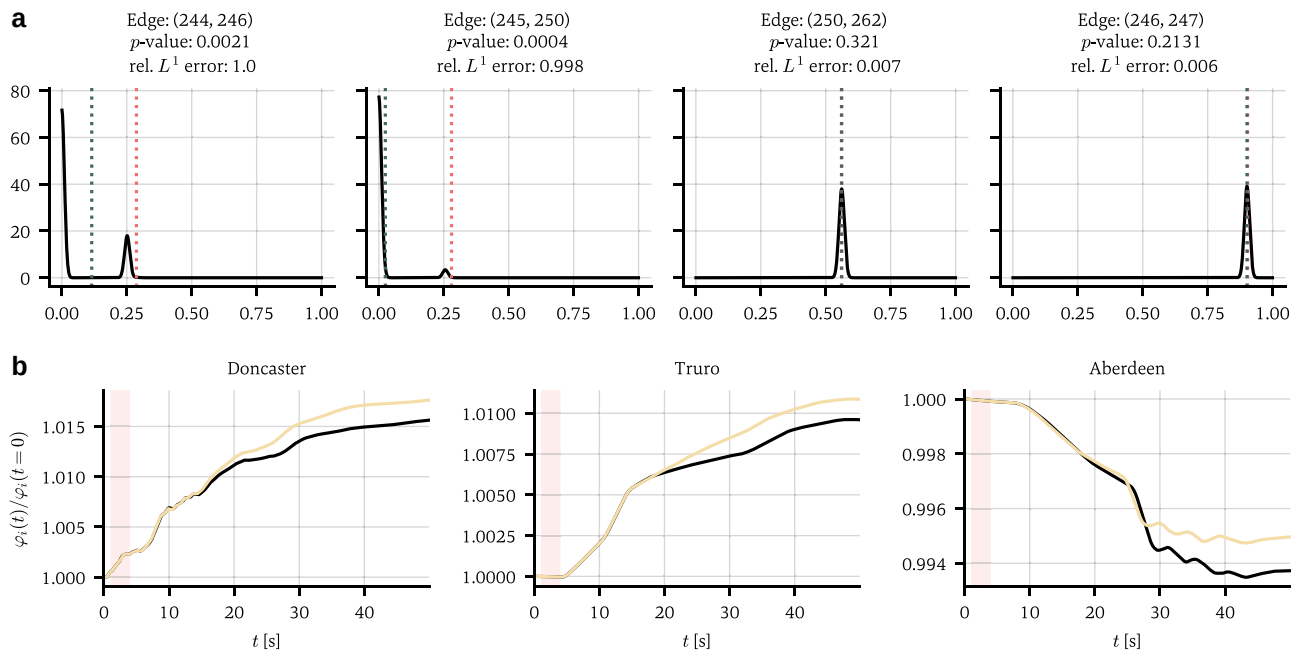

Fig. 3. Estimating the line failure location. a) The densities on the four edges with the highest relative prediction error $|\tilde{a}_{ij} - a^0_{ij}|/a^0_{ij}$ and their respective p-values for measuring the unperturbed value a^0_{ij} ($\tilde{a}_{ij}$ is the prediction mode). Red dotted lines indicate the values of the unperturbed network, green lines the expectation values of the distributions. The marginals are smoothed using a Gaussian kernel. We use a training set of length $L = 400$ steps, and the batch size is $B = 2$. CPU runtime: 24 min. b) True (black) and predicted network responses at three different locations in the network. The responses are each normalized to the value at $t = 0$. The shaded area represents the 400 time steps used to train the model. While the model is able to perfectly fit the response within the training range, it is not able to learn the full network from insufficient data, causing the time series to diverge for larger t.

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

The sizes W_j are governed by a system of M coupled logistic Stratonovich stochastic differential equations

$$dW_j = \epsilon W_j(D_j - \kappa W_j)\,dt + \sigma W_j \circ d\xi_j, \tag{7}$$

with given initial conditions $W_j(0)$, see Fig. 4a. α, β, κ, and ϵ are scalar parameters. Our goal is to infer the cost matrix $\mathbf{C} = (c_{ij})$ from observations of the time series $\mathbf{O}(t)$ and $\mathbf{W}(t)$. The model includes multiplicative noise with strength $\sigma \geq 0$, where the ξ_j are independent white noise processes and $\circ$ signifies Stratonovich integration (57). Crucially, the model depends nonlinearly on $\mathbf{C}$.

We apply this model to a previously studied dataset of economic activity in Greater London (15, 31). We use the ward-level household income from $N = 625$ wards for 2015 (54) and the retail floor space of the $M = 49$ largest commercial centers in London (53) as the initial origin zone and destination zone sizes, respectively, i.e. $\mathbf{O}(0)$ and $\mathbf{W}(0)$, and from this generate a synthetic time series using the parameters estimated in Ref. (31) for a high noise level of $\sigma = 0.14$. For the network $\mathbf{C}$, we use the Google Distance Matrix API[a] to extract the shortest travel time d_{ij} between nodes, using either public transport or driving. The network weights are derived in Ref. (58) as

$$c_{ij} = e^{-d_{ij}/\tau},$$

where the scale factor $\tau = \max_{i,j} d_{ij}$ ensures a unitless exponent.

We generate a synthetic time series of 10,000 time steps, from which we subsample 2,500 2-step windows, giving a total training set size of $L = 5{,}000$ time steps. This is to ensure we sample a sufficiently broad spectrum of the system's dynamics, thereby fully determining the inference problem and isolating the effect of the training noise. A hyperparameter sweep on synthetic data showed that using a neural network with 2 layers, 20 nodes per layer, and no bias yields optimal results. We use the hyperbolic tangent as the activation function on all layers except the last, where we use the standard sigmoid function (since the network is complete, there is no need to use the hard sigmoid as all edge weights are nonzero). To train the neural network, we use the simple loss function

$$J = \|\hat{\mathbf{T}}(\hat{\mathbf{A}}) - \mathbf{T}\|_2^2,$$

where $\hat{\mathbf{T}}$ and $\mathbf{T}$ are the predicted and true time series of destination zone sizes. Since the dynamics are invariant under scaling of the cost matrix $\mathbf{C} \to \lambda\mathbf{C}$, we normalize the row sums of the predicted and true networks, $\sum_j c_{ij} = 1$.

Figure 4c shows the inferred distribution $\rho(k)$ of the (weighted) origin zone node degrees $k_j = \sum_i c_{ij}$. The solid line is the maximum likelihood prediction, and the dotted line the true distribution. Even with a high level of noise, the model manages to accurately predict the underlying connectivity matrix, comprising over 30,000 weights, in under 5 min on a regular laptop CPU. Uncertainty on $P(k)$ is given by the standard deviation,

$$\mathbb{E}_{\hat{\mathbf{T}}}\left[P(k \mid \hat{\mathbf{T}}) - \hat{P}(k)\right]^2, \tag{8}$$

where $\hat{P}$ is the maximum likelihood estimator. As we will discuss in the last section, this method meaningfully captures the

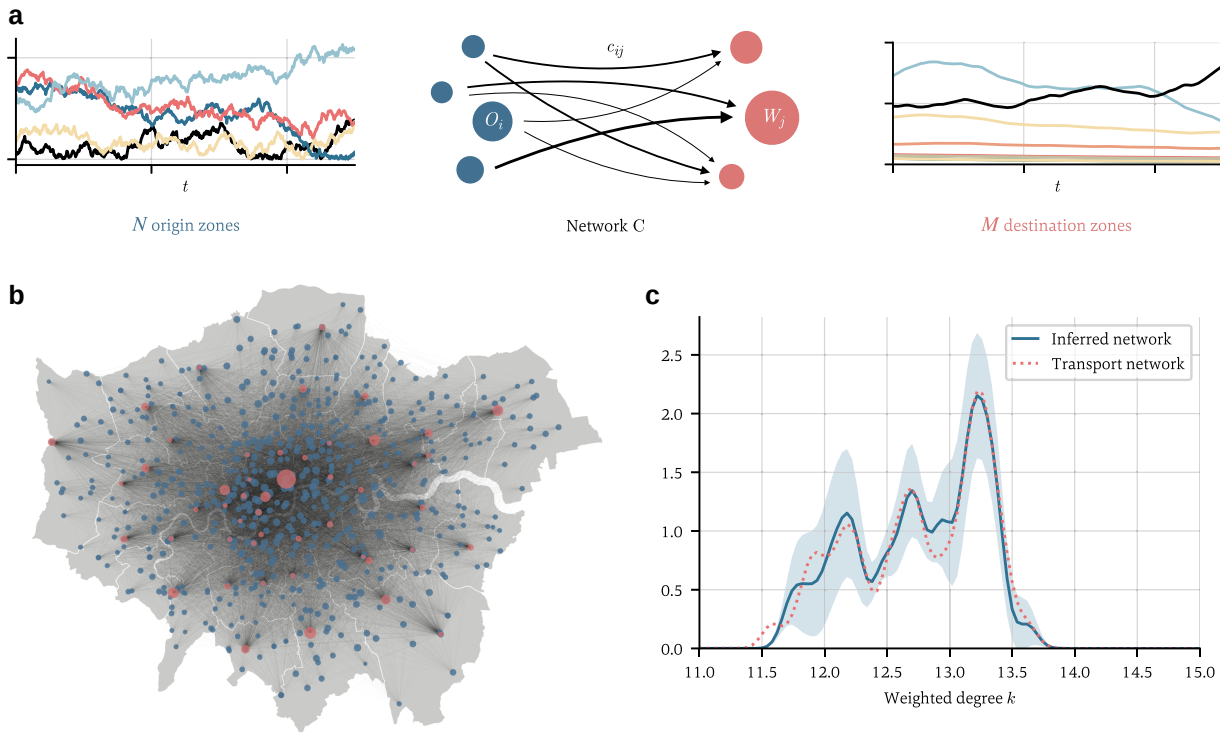

Fig. 4. Inferring economic cost networks. a) In the model, N origin zones (red) are connected to M destination zones (blue) through a weighted directed network. Economic demand flows from the origin zones to the destination zones, which supply the demand. We model the origin zones $O_i(t)$ as a Wiener process with diffusion coefficient $\sigma_O = 0.1$. The resulting cumulative demand at destination zone j is given by W_j. Note that the origin zone sizes fluctuate more rapidly than the destination zones, since there is a delay in the destination zones' response to changing consumer patterns, controlled by the parameter ϵ. We use the parameters as estimated in Ref. (31), $\alpha = 0.92$, $\beta = 0.54$, $\kappa = 8.3$, and set $\epsilon = 2$. b) The initial origin and destination zone sizes, given by the total household income of the $N = 629$ wards in London (blue nodes) and the retail floor space of $M = 49$ major centers (red nodes) (53, 54). The network is given by travel times as detailed in the text. Background map: Ref. (55). c) Predicted degree distribution (sold line) of the inferred network, for a high noise level of $\sigma = 0.14$, and 1 SD (shaded area), and the true distribution (dotted line). CPU runtime: 3 min 41 s.

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

uncertainty due to the noise in the data and the degree to which the problem is underdetermined.

Comparative performance analysis

We now analyze our method's performance, both in terms of prediction quality and computational speed, by comparing it to a Markov-chain Monte Carlo (MCMC) approach as well as a classical regression method, presented e.g. in Refs. (6, 59). As mentioned in the Introduction section, computationally efficient network learning methods have been developed for specific data structures; however, we compare our approach with MCMC and OLS since both are general in the types of data to which they are applicable.

Consider noisy Kuramoto dynamics,

$$\alpha \frac{\mathrm{d}^2\varphi_i}{\mathrm{d}t^2} + \frac{\mathrm{d}\varphi_i}{\mathrm{d}t} - \omega_i = \sum_j a_{ij} \sin(\varphi_j - \varphi_i) + \xi_i, \tag{9}$$

with ξ_i independent white noise processes with strength σ and ω_i the eigenfrequencies of the nodes. Given L observations of each node's dynamics, we can gather the left-hand side into a single vector $\mathbf{X}_i \in \mathbb{R}^{1\times L}$ for each node, and obtain N equations

$$\mathbf{X}_i = \mathbf{A}_i \times \mathbf{G}_i + \boldsymbol{\xi}_i, \quad i = 1, \ldots, N, \tag{10}$$

with $\mathbf{A}_i \in \mathbb{R}^{1\times N}$ the ith row of the adjacency matrix $\mathbf{A}$ and $\mathbf{G}_i \in \mathbb{R}^{N\times L}$ the L observations of the interaction terms $\sin(\varphi_j - \varphi_i)$, $j = 1, \ldots, N$. From this, we can then naturally estimate the ith row of $\mathbf{A}$ using OLS:

$$\hat{\mathbf{A}}_i = \underset{\gamma\in\mathbb{R}^{1\times N}}{\operatorname{argmin}} \|\mathbf{X}_i - \gamma\mathbf{G}_i\|_2^2 = \mathbf{X}_i\mathbf{G}_i^{\mathrm{T}}\left(\mathbf{G}_i\mathbf{G}_i^{\mathrm{T}}\right)^{-1}. \tag{11}$$

Given sufficiently many linearly independent observations, the Gram matrices $\mathbf{G}_i\mathbf{G}_i^{\mathrm{T}}$ will all be invertible; in the underdetermined case, a pseudoinverse can be used to approximate their inverses. As before, the diagonal of $\hat{\mathbf{A}}$ is manually set to 0.

In addition to regression, we also compare our method to a preconditioned Metropolis-adjusted Langevin (MALA) sampling scheme (27–29, 60), which constructs a Markov chain of sampled adjacency matrices $\hat{\mathbf{A}}$ by drawing proposals from the normal distribution

$$\hat{\mathbf{A}}^{i+1} \sim \mathcal{N}\left(\hat{\mathbf{A}}^{i} + \frac{\tau}{2}\lambda^{-1}\mathbf{P}\nabla J\left(\hat{\mathbf{A}}^{i} \mid \mathbf{T}\right), \tau\lambda^{-1}\mathbf{P}\right). \tag{12}$$

Here, $\tau > 0$ is the integration step size, $\mathbf{P} \in \mathbb{R}^{N^2\times N^2}$ is a preconditioner (note that we are reshaping $\hat{\mathbf{A}}$ into an N^2-dimensional vector), and $\lambda = \mathrm{tr}(\mathbf{P})/N^2$ is its average eigenvalue. Each proposal is accepted with probability

$$\eta = \min\left[1, \frac{\exp\left(-J\left(\hat{\mathbf{A}}^{i+1}\right)\right)q\left(\hat{\mathbf{A}}^{i+1} \mid \hat{\mathbf{A}}^{i}\right)}{\exp\left(-J\left(\hat{\mathbf{A}}^{i}\right)\right)q\left(\hat{\mathbf{A}}^{i} \mid \hat{\mathbf{A}}^{i+1}\right)}\right], \tag{13}$$

with the transition probability

$$q\left(\hat{\mathbf{A}}^{i+1} \mid \hat{\mathbf{A}}^{i}\right) \propto \exp\left(-\frac{1}{4\tau}\|\hat{\mathbf{A}}^{i+1} - \hat{\mathbf{A}}^{i} - \tau\nabla\log\pi(\hat{\mathbf{A}}^{I})\|_2^2\right). \tag{14}$$

We tune τ so that the acceptance ratio η converges to the optimum value of 0.57 (61).

We set the preconditioner $\mathbf{P}$ to be the inverse Fisher information covariance matrix

$$\mathbf{P}^{-1} = \mathbb{E}_{\hat{\mathbf{A}}}\left[\nabla J(\hat{\mathbf{A}}^{i})\nabla J(\hat{\mathbf{A}}^{i})^{\mathrm{T}}\right], \tag{15}$$

which has been shown to optimize the expected squared jump distance (29). The expectation value is calculated empirically over all samples drawn using the efficient algorithm given in Ref. (29). In all experiments, we employ a "warm start" by initializing the sampler close to the minimum of the problem. We found this to be necessary in such high dimensions (between 256 and 490,000) to produce decent results. Unlike the MCMC sampler, the neural network is initialized randomly.

Figure 5a and b shows our method's prediction accuracy alongside that of OLS regression and preconditioned MALA on synthetic Kuramoto data; the accuracy here is defined as the L^1 error

$$\|\hat{\mathbf{A}} - \mathbf{A}\|_1 = \sum_{i,j} |\hat{a}_{ij} - a_{ij}|, \tag{16}$$

where $\hat{\mathbf{A}}$ is the mode of the posterior. In Fig. 5a, the accuracy is shown as a function of the noise σ on the training data. We generate enough data to ensure the likelihood function is unimodal. For the practically noiseless case of $\sigma < 10^{-5}$, the regression scheme on average outperforms the neural approach; however, even for very low noise levels $\sigma \geq 10^{-5}$ and above, the neural approach proves far more robust, outperforming OLS by up to one order of magnitude and maintaining its prediction performance up to low noise levels of $\sigma \leq 10^{-3}$. Meanwhile, we find that in the low- to mid-level noise regime, the neural scheme approximates the mode of the distribution by between 1 and 2 orders of magnitude more accurately than the Langevin sampler. For high levels of noise ($\sigma > 10^{-2}$), the performances of the neural and MALA schemes converge. These results hold both for first-order ($\alpha = 0$) and second-order Kuramoto dynamics [3]; in the second-order case, the neural method begins outperforming OLS at even lower levels of σ than in the first-order case, though the improvement is not as significant (cf. Fig. S6).

In Fig. 5b, we show the accuracy as a function of the convexity of the loss function. In general, it is hard to quantify the convexity of J, since we do not know how many networks fit the equation at hand. However, when the dynamics are linear in the adjacency matrix $\mathbf{A}$, we can do so using the Gram matrices of the observations of each node i, $\mathbf{G}_i\mathbf{G}_i^{\mathrm{T}}$, where we quantify the (non)convexity of the problem by the minimum rank of all the Gram matrices,

$$\mathfrak{c} := \min_i \mathrm{rk}\left(\mathbf{G}_i\mathbf{G}_i^{\mathrm{T}}\right). \tag{17}$$

The problem is fully determined if $\mathfrak{c} = N - 1$ and all Gram matrices are invertible. As shown, regression is again more accurate when the problem is close to fully determined; however, as $\mathfrak{c}$ decreases, the accuracy quickly drops, with the neural scheme proving up to an order of magnitude more accurate. Meanwhile, the MCMC scheme is consistently outperformed by the neural scheme, though it too eclipses regression for $\mathfrak{c} < 0.75$. In summary, regression is only viable for the virtually noiseless and fully determined case, while the neural scheme maintains good prediction performance even in the noisy and highly underdetermined case (see also Fig. 5d and e).

In Fig. 5c, we show compute times to obtain 100 samples for both the neural and MALA schemes. The complexity of the neural scheme is $\mathcal{O}(n_{\mathrm{E}} \times LN^2)$, with n_{E} the number of training epochs. This is because each epoch of the model equation requires $\mathcal{O}(LN^2)$ operations for the vector–matrix multiplication in Eq. 11, and $\mathcal{O}(LN^2/B)$ for the stochastic gradient descent update, where we are holding L/B constant to ensure comparability. As is visible, the average L^1 error per edge weight remains constant over N, showing that the number of epochs required to achieve a given node-averaged prediction

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

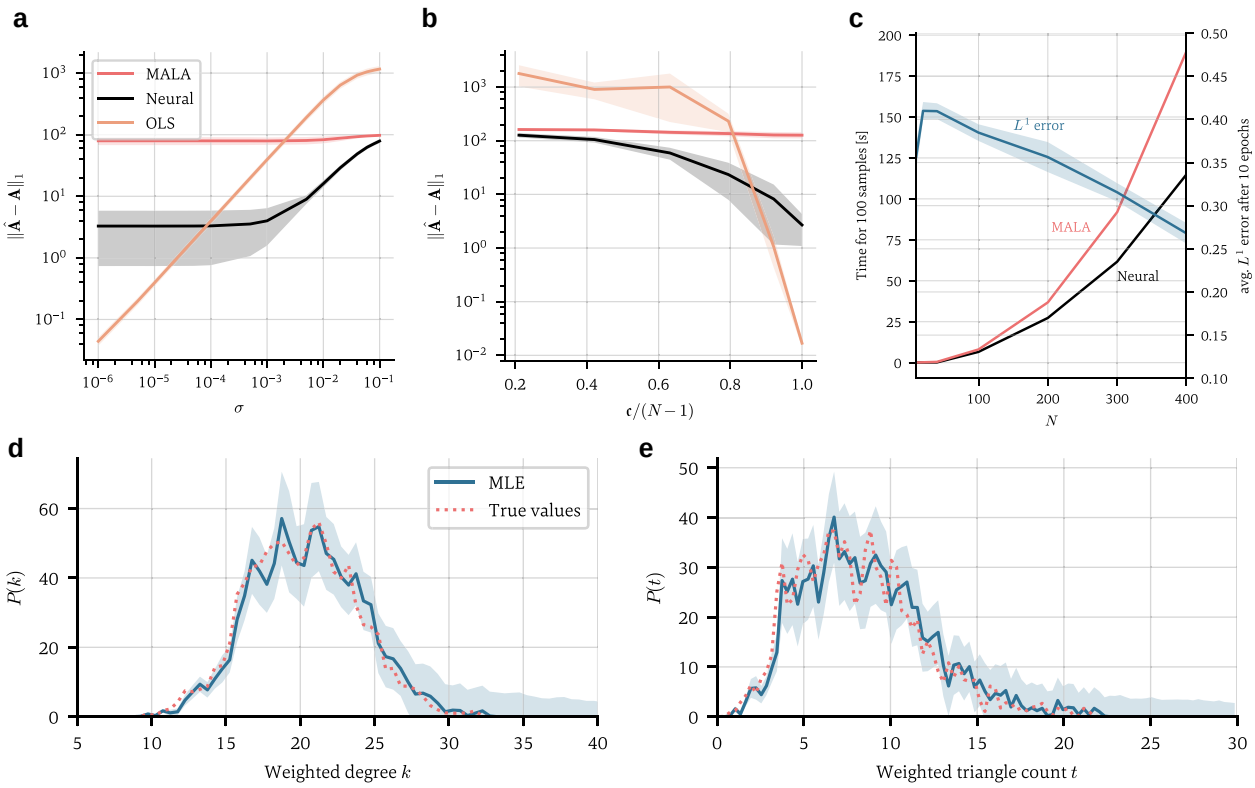

Fig. 5. Computational performance analysis. a) L^1 prediction error (Eq. 16) of the neural scheme, the preconditioned Metropolis-adjusted Langevin sampler, and OLS regression as a function of the noise variance σ on the training data. For very high noise levels, the training data are essentially pure noise, and the prediction errors begin to plateau. First-order Kuramoto dynamics are used ($\alpha = 0$), though these results also hold for second-order dynamics (cf. Fig. S6). Enough data are used to ensure full invertibility of the Gram matrix ($\mathfrak{c} = 1$). b) The L^1 accuracy as a function of the convexity $\mathfrak{c}$ of the loss function (Eq. 17). c) Compute times for 10 epochs, or 100 samples, of the neural scheme and the preconditioned Metropolis-adjusted Langevin sampler, averaged over 10 runs. The shaded areas show one standard deviation. On the right axis, the average L^1 prediction error of the neural scheme $\frac{1}{N}\|\hat{\mathbf{A}} - \mathbf{A}\|_1$ after 10 epochs is shown, which remains fairly constant as a function of N, showing that the number of gradient descent steps required to achieve a given average prediction error does not depend on N. d) Predicted degree distribution and e) triangle distribution of an inferred network with $N = 1{,}000$ nodes, trained on first-order noisy Kuramoto data ($\sigma = 0.001$). The blue shaded areas indicate one standard deviation, and the dotted lines are the true distributions. CPU runtime: 1 h 3 min.

accuracy is independent of N. The preconditioned MALA scheme is considerably slower, due to the computational cost of calculating the preconditioner and the Metropolis–Hastings rejection step.

Lastly, Figs. 5d and e show the estimated weighted degree and triangle distributions of a large graph with 1,000 nodes, or 1 million edge weights to be estimated, for noisy training data. The number of weighted, undirected triangles on each node i is given by $\frac{1}{2}\sum_{jk} a_{ij}a_{jk}a_{ki}$. The model robustly finds the true adjacency matrix, and we again quantify uncertainty on the prediction using the standard deviation (Eq. 8). Estimating a network with 1,000 nodes on a standard laptop CPU took about 1 h, which reduces to 6 min when using a GPU. Most high-performance network inference techniques demonstrate their viability on graphs with at most this number of nodes, e.g. ConNIe (19) and NetINF (21). In Ref. (19), the authors state that graphs with 1,000 nodes can typically be inferred from cascade data in under 10 min on a standard laptop. Similarly, the authors of NetINF (21) state that it can infer a network with 1,000 nodes in a matter of minutes, though this algorithm does not infer edge weights, only the existence of edges, and neither technique provides uncertainty quantification.

Quantifying uncertainty

There are two sources of uncertainty when inferring adjacency matrices: the nonconvexity of the loss function J and the noise σ on the data. In Fig. 6a, we show the expected Hellinger error

$$\frac{1}{2}\mathbb{E}_{\hat{\mathbf{T}}}\int\left[\sqrt{P(x\,|\,\hat{\mathbf{T}})} - \sqrt{\hat{P}(x)}\right]^2 dx \tag{18}$$

on the predicted degree distribution as a function of $\mathfrak{c}$. As is visible, the error on the distribution decreases as $\mathfrak{c}$ tends to its maximum value of $N - 1$. For $\mathfrak{c} = N - 1$, some residual uncertainty remains due to the uncertainty on the neural network parameters $\boldsymbol{\theta}$.

In Fig. 6b and c, we show the expected Hellinger error (Eq. 18) on the maximum likelihood estimator $\hat{P}$ as a function of σ, for both the degree and triangle distributions, i.e. $x \in \{k, t\}$. In addition, we also show the behavior of the expected relative entropy

$$\mathbb{E}_{\hat{\mathbf{T}}}\int P(x\,|\,\hat{\mathbf{T}})\log\left(\frac{P(x\,|\,\hat{\mathbf{T}})}{\hat{P}(x)}\right)dx \tag{19}$$

and the total SD

$$s^2 = \int \mathbb{E}_{\hat{\mathbf{T}}}\left[P(x\,|\,\hat{\mathbf{T}}) - \hat{P}(x)\right]^2 dx. \tag{20}$$

All three metrics reflect the noise on the training data, providing similarly behaved, meaningful uncertainty quantification. As the noise tends to 0, some residual uncertainty again remains,

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

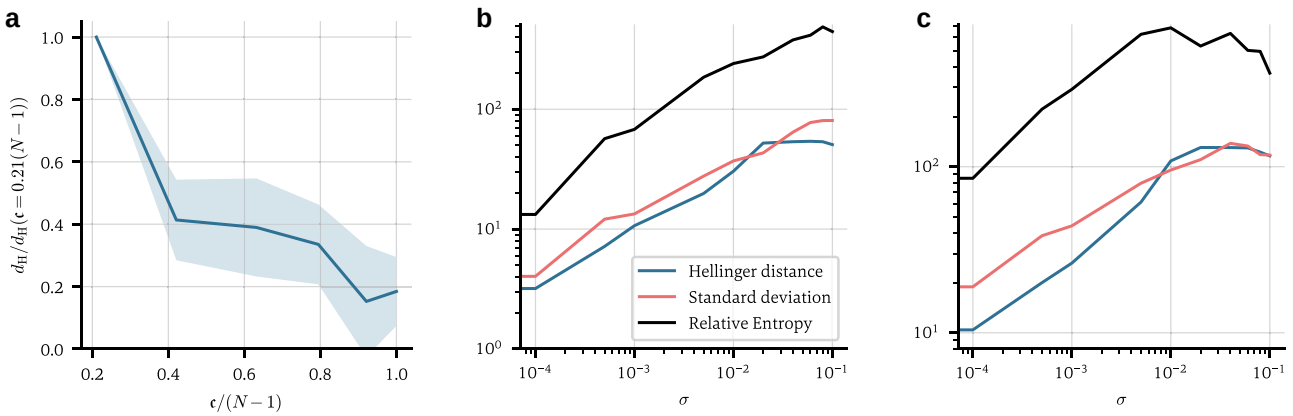

Fig. 6. Quantifying the two types of uncertainty: a) Hellinger error (Eq. 18) on the degree distribution $P(k)$ as a function of $\mathfrak{c}$ (Eq. 17) in the noiseless case. The error is normalized to the value at $\mathfrak{c} = 0.21(N-1)$. As $\mathfrak{c}$ increases, the error on the prediction decreases almost linearly. We run the model from 10 different initializations and average over each (shaded area: SD). b and c) Prediction uncertainty due to noise in the data. The expected Hellinger error (Eq. 18) and expected relative entropy (Eq. 19) to the maximum likelihood estimate, as well as the total SD s (Eq. 20) for the degree distribution $P(k)$ and triangle distribution $P(t)$ as a function of the noise σ on the data are shown. Each line is an average over 10 different initializations. In all cases, training was conducted on synthetic, first-order Kuramoto data (Eq. 9, with $\alpha = 0$).

while for very high noise levels, the uncertainty begins to plateau. Our method thus manages to capture the uncertainty arising from both sources: the nonconvexity of J and the noise σ on the data.

Discussion

In this work, we have demonstrated a performative method to estimate network adjacency matrices from time series data. We showed its effectiveness at correctly and reliably inferring networks in a variety of scenarios: convex and nonconvex cases, low- to high-noise regimes, and equations that are both linear and nonlinear in **A**. We were able to reliably infer power line failures in the national power grid of Great Britain, and the connectivity matrix of an economic system covering all of Greater London. We showed that our method is well able to handle inference of hundreds of thousands to a million edge weights, while simultaneously giving uncertainty quantification that meaningfully reflects both the nonconvexity of the loss function as well as the noise on the training data. Our method is significantly more accurate than MCMC sampling and outperforms OLS regression in all except the virtually noiseless and fully determined cases. This is an important improvement since large amounts of data are typically required to ensure the network inference problem is fully determined, which may often not be available, as suggested in the power grid study. Unlike regression, our method also naturally extends to the case of nonlinear dynamics. In conjunction with our previous work (31), we have now also demonstrated the viability of using neural networks for parameter calibration in both the low- and high-dimensional case. Our method is simple to implement as well as highly versatile, giving excellent results across a variety of problems. All experiments in this work were purposefully conducted on a standard laptop CPU, typically taking on the order of minutes to run.

Many lines for future research open up from this work. First, a thorough theoretical investigation of the method is warranted, establishing rigorous convergence guarantees and bounds on the error of the posterior estimate. Another direction is further reducing the amount of data required to accurately learn parameters, and in future research the authors aim to address the question of learning system properties from observations of a single particle trajectory at the mean-field limit (62, 63). In this work, we have also not considered the impact of the network topology on the prediction performance, rather focusing on the physical dynamics of the problem. An interesting question is to what degree different network structures themselves are amenable to or hinder the learning process.

Over the past decade much work has been conducted into graph neural architectures (64, 65), the use of which may further expand the capabilities of our method. More specialized architectures may prove advantageous for different (and possibly more difficult) inference tasks, though we conducted a limited number of experiments with alternatives (e.g. autoencoders, cf. Fig. S4) and were unable to find great performance improvements. Finally, one drawback of our proposed method in its current form is it that it requires differentiability of the model equations in the parameters to be learned; future research might aim to develop a variational approach to expand our method to weakly differentiable settings.

Note

[a] developers.google.com/maps/documentation/distance-matrix.

Acknowledgments

The authors are grateful to Dr Andrew Duncan (Imperial College London) for fruitful discussions on power grid dynamics, and to the anonymous reviewers for their helpful comments during the peer review process.

Supplementary Material

Supplementary material is available at *PNAS Nexus* online.

Funding

T.G. was funded by the University of Cambridge School of Physical Sciences VC Award via DAMTP and the Department of Engineering, and supported by EPSRC grants EP/P020720/2 and EP/R018413/2. The work of G.A.P. was partially funded by EPSRC grant EP/P031587/1. M.G. was supported by EPSRC grants EP/T000414/1, EP/R018413/2, EP/P020720/2, EP/R034710/1, EP/R004889/1, and a Royal Academy of Engineering Research Chair.

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

Author Contributions

T.G., G.A.P., and M.G. designed and performed the research and wrote the paper. T.G. wrote the code and performed the numerical experiments.

Preprints

This manuscript was posted on a preprint: arxiv.org/abs/2303.18059.

Data Availability

Code and synthetic data can be found under https://github.com/ThGaskin/NeuralABM. It is easily adaptable to new models and ideas. The code uses the `utopya` package (utopia-project.org, utopya.readthedocs.io/en/latest) (66, 67) to handle simulation configuration and efficiently read, write, analyze, and evaluate data. This means that the model can be run by modifying simple and intuitive configuration files, without touching code. Multiple training runs and parameter sweeps are automatically parallelized. The neural core is implemented using `pytorch` (pytorch.org). All datasets have been made available, together with the configuration files needed to reproduce the plots. Detailed instructions are provided in supplementary material and the repository.

References

1 Simard SW, *et al.* 2012. Mycorrhizal networks: mechanisms, ecology and modelling. *Fungal Biol Rev*. 26(1):39–60.
2 Hettenhausen C, *et al.* 2017. Stem parasitic plant *Cuscuta australis* (dodder) transfers herbivory-induced signals among plants. *Proc Natl Acad Sci U S A*. 114(32):E6703–E6709.
3 Brockmann D, Helbing D. 2013. The hidden geometry of complex, network-driven contagion phenomena. *Science*. 342(6164): 1337–1342.
4 Molkenthin N, Schröder M, Timme M. 2020. Scaling laws of collective ride-sharing dynamics. *Phys Rev Lett*. 125(24):248302.
5 Simonsen I, Buzna L, Peters K, Bornholdt S, Helbing D. 2008. Transient dynamics increasing network vulnerability to cascading failures. *Phys Rev Lett*. 100(21):218701.
6 Shandilya SG, Timme M. 2011. Inferring network topology from complex dynamics. *New J Phys*. 13(1):013004.
7 Stelzl U, *et al.* 2005. A human protein–protein interaction network: a resource for annotating the proteome. *Cell*. 122(6): 957–968.
8 Proulx SR, Promislow DEL, Phillips PC. 2005. Network thinking in ecology and evolution. *Trends Ecol Evol*. 20(6):345–353.
9 Allesina S, Alonso D, Pascual M. 2008. A general model for food web structure. *Science*. 320(5876):658–661.
10 Tegnér J, Yeung MKS, Hasty J, Collins JJ. 2003. Reverse engineering gene networks: integrating genetic perturbations with dynamical modeling. *Proc Natl Acad Sci U S A*. 100(10):5944–5949.
11 Palsson BO. 2006. *Systems biology: properties of reconstructed networks*. New York (NY): Cambridge University Press.
12 Sarmah D. 2022. Network inference from perturbation time course data. *NPJ Syst Biol Appl*. 8(1):42.
13 Shen B, Coruzzi G, Shasha D. 2023. EnsInfer: a simple ensemble approach to network inference outperforms any single method. *BMC Bioinf*. 24(1):114.
14 Batty M, Milton R. 2021. A new framework for very large-scale urban modelling. *Urban Stud*. 58(15):3071–3094.
15 Ellam L, Girolami M, Pavliotis GA, Wilson A. 2018. Stochastic modelling of urban structure. *Proc R Soc A: Math Phys Eng Sci*. 474(2213):20170700.
16 Vicario MD, *et al.* 2016. The spreading of misinformation online. *Proc Natl Acad Sci U S A*. 113(3):554–559.
17 Aral S, Muchnik L, Sundararajan A. 2013. Engineering social contagions: optimal network seeding in the presence of homophily. *Netw Sci*. 1(2):125–153.
18 Vosoughi S, Roy D, Aral S. 2018. The spread of true and false news online. *Science*. 359(6380):1146–1151.
19 Myers SA, Leskovec J. 2010. On the convexity of latent social network inference. In: Proceedings of the 23rd International Conference on Neural Information Processing Systems—Volume 2, NIPS'10. Red Hook (NY): Curran Associates Inc. p. 1741–1749.
20 Gomez-Rodriguez M, Balduzzi D, Schölkopf B. 2011. Uncovering the temporal dynamics of diffusion networks. In: Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML'11. Madison (WI): Omnipress. p. 561–568.
21 Gomez-Rodriguez M, Leskovec J, Krause A. 2012. Inferring networks of diffusion and influence. *ACM Trans Knowl Discov Data*. 5(4):1–37.
22 Makarov VA, Panetsos F, de Feo O. 2005. A method for determining neural connectivity and inferring the underlying network dynamics using extracellular spike recordings. *J Neurosci Methods*. 144(2):265–279.
23 Van Bussel F. 2011. Inferring synaptic connectivity from spatio-temporal spike patterns. *Front Comput Neurosci*. 5:1662.
24 Meinshausen N, Bühlmann P. 2006. High-dimensional graphs and variable selection with the Lasso. *Ann Stat*. 34(3):1436–1462.
25 Yuan M, Lin Y. 2007. Model selection and estimation in the Gaussian graphical model. *Biometrika*. 94(1):19–35.
26 Timme M, Casadiego J. 2014. Revealing networks from dynamics: an introduction. *J Phys A Math Theor*. 47(34):343001.
27 Girolami M, Calderhead B. 2011. Riemann manifold Langevin and Hamiltonian Monte Carlo methods. *J R Stat Soc Series B Stat Methodol*. 73(2):123–214.
28 Li C, Chen C, Carlson D, Carin L. 2016. Preconditioned stochastic gradient Langevin dynamics for deep neural networks. In: Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, AAAI'16. AAAI Press. p. 1788–1794.
29 Titsias MK. 2023. Optimal preconditioning and fisher adaptive Langevin sampling, arXiv, arXiv:2305.14442, preprint: not peer reviewed.
30 Göttlich S, Totzeck C. 2021. Parameter calibration with stochastic gradient descent for interacting particle systems driven by neural networks. *Math Control Signals, Syst*. 34(1):185–214.
31 Gaskin T, Pavliotis GA, Girolami M. 2023. Neural parameter calibration for large-scale multi-agent models. *Proc Natl Acad Sci U S A*. 120(7):e2216415120.
32 Lee J, *et al.* 2018. Deep neural networks as Gaussian processes. In: International Conference on Learning Representations (ICLR) 2018. https://openreview.net/forum?id=B1EA-M-0Z.
33 Matthews AGDG, Rowland M, Hron J, Turner RE, Ghahramani Z. 2018. Gaussian process behaviour in wide deep neural networks, arXiv, arXiv:1804.11271, preprint: not peer reviewed.
34 Novak R, *et al.* 2019. Bayesian deep convolutional networks with many channels are Gaussian processes. In: International Conference on Learning Representations (ICLR), 2019. https://openreview.net/forum?id=B1g30j0qF7.
35 Kingma DP, Welling M. 2013. Auto-encoding variational Bayes, arXiv, arXiv:1312.6114, preprint: not peer reviewed.

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025

36 Blundell C, Cornebise J, Kavukcuoglu K, Wierstra D. 2015. Weight uncertainty in neural networks. In: Proceedings of the 32nd international conference on Machine Learning, JMLR.org. ICML15. p. 1613–1622. https://dl.acm.org/doi/proceedings/10.5555/3045118.
37 Gal Y, Ghahramani Z. 2016. Dropout as a Bayesian approximation: representing model uncertainty in deep learning. In: Proceedings of the 33rd International Conference on Machine Learning. Vol. 48. JMLR. p. 1050–1059.
38 Stuart AM. 2010. Inverse problems: a Bayesian perspective. *Acta Numer*. 19:451–559.
39 Kingma DP, Ba J. 2014. Adam: a method for stochastic optimization, arXiv, arXiv:1412.6980, preprint: not peer reviewed .
40 Kuramoto Y. 1975. Self-entrainment of a population of coupled non-linear oscillators. In: Araki H, editor. *International Symposium on Mathematical Problems in Theoretical Physics*. Berlin: Springer. p. 420–422.
41 Filatrella G, Nielsen AH, Pedersen NF. 2008. Analysis of a power grid using a Kuramoto-like model. *Eur Phys J B*. 61(4):485–491.
42 Rohden M, Sorge A, Timme M, Witthaut D. 2012. Self-organized synchronization in decentralized power grids. *Phys Rev Lett*. 109:064101.
43 Nishikawa T, Motter AE. 2015. Comparative analysis of existing models for power-grid synchronization. *New J Phys*. 17(1):015012.
44 Choi Y-P, Li Z. 2019. Synchronization of nonuniform Kuramoto oscillators for power grids with general connectivity and dampings. *Nonlinearity*. 32(2):559–583.
45 Department for Business, Energy and Industrial Strategy. Digest of UK Energy Statistics 5: Electricity. Jul 2022.
46 National Grid. Transmission Network Shapefiles. Jan 2023. https://www.nationalgrid.com/electricity-transmission/network-and-infrastructure/network-route-maps.
47 SP Energy Networks. Transmission Network GIS Shapefiles. 2023. https://www.spenergynetworks.co.uk/pages/utility_map_viewer.aspx.
48 Scottish and Southern Electricity Networks. Transmission Network GIS Shapefiles. Feb 2023. https://www.ssen.co.uk/globalassets/library/connections—useful-documents/network-maps/5-gis-guide-shape-files-v1.pdf.
49 Office for National Statistics 2023: Westminster Parliamentary Constituencies (December 2022) Boundaries UK BFC, 2023. https://geoportal.statistics.gov.uk/search?q=BDY_PCON%3BDEC_2022&sort=Title|title|asc.
50 Hardsigmoid–Pytorch Documentation. Hardsigmoid, 2023. https://pytorch.org/docs/stable/generated/torch.nn.Hardsigmoid.html.
51 Hardsigmoid–Tensorflow Documentation. Hardsigmoid, 2023. https://www.tensorflow.org/api_docs/python/tf/keras/activations/hard_sigmoid.
52 Basiri F, Casadiego J, Timme M, Witthaut D. 2018. Inferring power-grid topology in the face of uncertainties. *Phys Rev E*. 98:012305.
53 Greater London Authority. 2017. Health Check Report, 2017. https://data.gov.uk/dataset/2a50ca67-954a-4f22-91d8-d3dfe9116143/london-town-centre-health-check-analysis-report.
54 Greater London Authority. 2015. Ward Profiles and Atlas, 2015. https://data.london.gov.uk/dataset/ward-profiles-and-atlas.
55 Greater London Authority. Statistical Boundary Files for London, 2011. https://data.london.gov.uk/dataset/statistical-gis-boundary-files-london.
56 Harris B, Wilson AG. 1978. Equilibrium values and dynamics of attractiveness terms in production-constrained spatial-interaction models. *Environ Plan A: Economy Space*. 10(4):371–388.
57 Pavliotis GA. 2014. *Stochastic processes and applications*. New York. Springer.
58 Wilson AG. 1967. A statistical theory of spatial distribution models. *Transp Res*. 1(3):253–269.
59 Timme M. 2007. Revealing network connectivity from response dynamics. *Phys Rev Lett*. 98(22):224101.
60 Chewi S, *et al.* 2021. Optimal dimension dependence of the Metropolis-adjusted Langevin algorithm. In: Belkin M, Kpotufe S, editors. Proceedings of Thirty Fourth conference on Learning Theory. Vol. 134. Proceedings of Machine Learning Research. p. 1260–1300. https://proceedings.mlr.press/v134/chewi21a.html.
61 Roberts GO, Rosenthal JS. 2002. Optimal scaling of discrete approximations to Langevin diffusions. *J R Stat Soc Series B Stat Methodol*. 60(1):255–268.
62 Pavliotis GA, Zanoni A. 2022. A method of moments estimator for interacting particle systems and their mean field limit, arXiv, arXiv:2212.00403, preprint: not peer reviewed.
63 Zagli N, Pavliotis GA, Lucarini V, Alecio A. 2023. Dimension reduction of noisy interacting systems. *Phys Rev Res*. 5:013078.
64 Bronstein MM, Bruna J, LeCun Y, Szlam A, Vandergheynst P. 2017. Geometric deep learning: going beyond Euclidean data. *IEEE Signal Process Mag*. 34(4):18–42.
65 Wu L, Cui P, Pei J, Zhao L. 2022. *Graph neural networks: foundations, frontiers, and applications*. Singapore: Springer.
66 Riedel L, Herdeanu B, Mack H, Sevinchan Y, Weninger J. 2020. Utopia: a comprehensive and collaborative modeling framework for complex and evolving systems. *J Open Source Softw*. 5(53):2165.
67 Sevinchan Y, Herdeanu B, Traub J. 2020. dantro: a python package for handling, transforming, and visualizing hierarchically structured data. *J Open Source Softw*. 5(52):2316.

Downloaded from https://academic.oup.com/pnasnexus/article/3/4/pgae063/7604085 by guest on 29 May 2025